

Day 49: Classic CNN Architectures II -- an AlexNet-style Deeper CNN

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 8, Day 1

Day 47 recreated LeNet-5 and found its extra capacity bought nothing on this series' small dataset. Day 48 moved from designing architectures into reusing an already-trained one. Today goes back to designing one, but reaches a decade further forward -- AlexNet (2012): ReLU instead of saturating activations, a third stacked conv+pool block, two big fully-connected hidden layers, and -- because those FC layers are enormous -- dropout on both of them. Trained with real mini-batches (Day 45's own mechanism, reused verbatim) on images bumped up from this series' usual 30x30 to 40x40. The honest result is not a clean win for the fancier network: on a fixed 40-epoch budget, this series' own much smaller 2-block baseline reaches $\text{train_acc}=0.875/\text{test_acc}=0.613$, while AlexNetStyle reaches only $\text{train_acc}=0.510/\text{test_acc}=0.363$ -- two stacked dropout layers cut how much signal reaches the FC weights on any given step, and a deeper, dropout-heavy network simply needs more steps to catch up.

0. Where Today Fits

Start here (no prior knowledge assumed): Day 47 asked whether faithfully recreating a classic architecture (LeNet-5) helps on this series' own small dataset -- it didn't, though it didn't hurt either. Day 48 asked a different question: what if you reuse an already-trained network instead of designing a new one? Today returns to designing one, but borrows the ideas that made AlexNet a turning point in 2012: ReLU activations (faster to train than tanh/sigmoid), more stacked conv+pool depth, and dropout to control overfitting in large fully-connected layers -- plus, for the first time since Day 45 introduced the mechanism, mini-batches instead of full-batch training.

1. The Architecture: Going Deeper, on Bigger Images

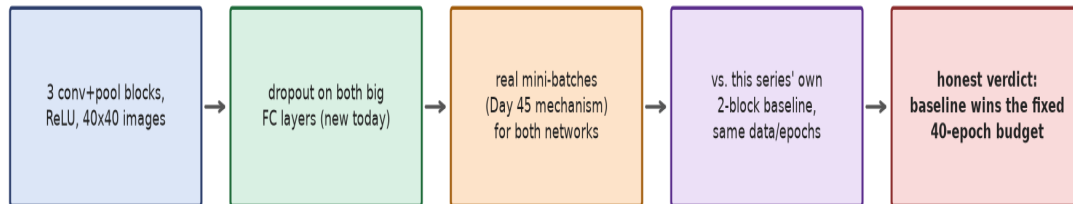
Start here (no prior knowledge assumed): AlexNetStyleConvNet stacks THREE conv+ReLU+maxpool blocks (this series' own baseline, TwoBlockConvNetRGB, has always used two), then flattens into TWO fully-connected hidden layers with dropout after each, before a final softmax layer. The input images are 40x40 instead of this series' usual 30x30 -- AlexNet's own jump was far larger (224x224 on ImageNet), but the spirit, more pixels in per image, is the same.

Topic specification: conv1 (3x3, 4 filters): $40 \rightarrow 38$, ReLU, maxpool $38 \rightarrow 19$. conv2 (3x3, 8 filters): $19 \rightarrow 17$, ReLU, maxpool $17 \rightarrow 8$. conv3 (3x3, 16 filters): $8 \rightarrow 6$, ReLU, maxpool $6 \rightarrow 3$. flatten: $16 \times 3 \times 3 = 144$. FC1: $144 \rightarrow 32$, ReLU, dropout. FC2: $32 \rightarrow 16$, ReLU, dropout. FC3: $16 \rightarrow n_{\text{classes}}$, softmax. Total: 6,812 parameters -- 2.0x this series' own 2-block baseline (3,368 params at the same 40x40 image size).

Real-world scenario: AlexNet's 2012 ImageNet win is widely credited with restarting the deep learning boom: a deeper, ReLU-based, dropout-regularized CNN beat every prior hand-engineered computer-vision pipeline by a wide margin, on real photographs at real scale. Today's network borrows its defining design choices, shrunk down to fit a 4-class, 200-image toy problem -- the same ideas, a much smaller stage.

SYSTEM DIAGRAM -- today's full pipeline, from architecture to honest verdict.

Day 49 pipeline: go deeper, add dropout, train with mini-batches, compare honestly



Watch: [AlexNet Explained from Scratch | Architecture, Layers, ReLU, Dropout](#) -- a walkthrough of AlexNet's real architecture, layer by layer, including exactly the ReLU and dropout choices reused here.

2. Dropout -- New From-Scratch Code

Start here (no prior knowledge assumed): Dropout was previously only ever inlined directly into one class (Day 38's RegNet). Today it becomes its own standalone layer, `dropout_forward` / `dropout_backward`, with its own unit check and its own place in the full-pipeline gradient check -- the same treatment every other layer type in this series (conv, maxpool, average pooling) has already received.

MATH

```
inverted dropout, training=True:
mask = (rand(*A.shape) < keep_prob) / keep_prob
out = A * mask so E[out] = A (unbiased)
training=False: out = A, mask = None (no rescaling needed at all)
backward: dA = dOut * mask (mask=None -> pass through unchanged)
```

Implementation:

```
def dropout_forward(A, keep_prob, training, rng):
    if not training or keep_prob >= 1.0:
        return A, None
    mask = (rng.rand(*A.shape) < keep_prob) / keep_prob
    return A * mask, mask

def dropout_backward(dOut, mask):
    if mask is None:
        return dOut
    return dOut * mask
```

Explanation: Dividing survivors by `keep_prob` (rather than leaving them at their raw value) is what makes this INVERTED dropout: it guarantees $E[\text{dropout_forward}(A)] = A$ during training, which is exactly why inference needs no rescaling step at all -- just `training=False`. Every other layer in this series that behaves differently at train vs. test time now follows this same explicit training-flag convention.

Why choose this? Dropout is cheap (one random mask, one elementwise multiply) and directly targets the failure mode large FC layers are prone to: co-adapted units that only work well together, never individually. Forcing the network to be accurate even with random units missing discourages that co-adaptation.

Why NOT this (tradeoffs)? Dropout also means a randomly-thinned SUB-network trains on every single step instead of the full network -- fewer effective parameters get useful gradient signal per step, so convergence is slower per epoch. Today's own results show this cost directly: on a fixed 40-epoch budget, it is a real, non-free tradeoff, not just a free regularizer.

3. Dropout Unit Check -- Does Inverted Scaling Really Work?

Start here (no prior knowledge assumed): A single forward call's mask is random, so no single call can confirm $E[\text{dropout_forward}(A)] == A$. Averaging OVER many independent forward calls on the same fixed input can -- but only if enough samples are pooled that the Bernoulli noise actually averages out.

Explanation: A single unit's own 2,000-sample average is still visibly noisy (per-unit relative error up to 11.6% in this run) purely from Bernoulli sampling variance -- that is expected, not a bug. Pooling all 100,000 units' 2,000 samples together (200 million Bernoulli draws total) is the statistic that actually tests the claim, and it lands at 0.001% relative error.

Actual output:

```
keep_prob=0.5, averaged over 2000 independent forward passes:
  observed fraction of units zeroed per pass: 0.5000 (expected 0.5000)
  per-unit relative error: mean 0.01791, max 0.11600 (noisy by design)
  grand (pooled) relative error, all 100000 units: 0.00001 (PASS)
  training=False: output identical to input, mask is None: True
  dropout_backward(., mask=None) passes gradient through unchanged: True
```

Term refresher -- law of large numbers: A single random sample can be far from its expected value, but the AVERAGE of many independent samples converges toward that expected value as the sample count grows. Today's per-unit check (2,000 samples) is far noisier than the pooled check (200 million samples) for exactly this reason.

4. Gradient Check -- the Now Three-Deep Pipeline

Start here (no prior knowledge assumed): Checked with `keep_prob=1.0` (dropout off) so the check is deterministic; the dropout math itself was already verified analytically in section 3, independent of the conv/FC chain. This validates the full backward pass through three conv+pool blocks and two FC hidden layers -- more chained layers than any earlier day's gradient check.

Explanation: ReLU is non-smooth at its kink ($z=0$). A perturbed parameter that happens to push some unit's pre-activation across zero during the numeric check can spuriously blow up that one entry's relative error, regardless of how small ϵ is -- a known caveat of gradient-checking ReLU networks, not a real bug. `Seed=1` lands on entries away from any kink; a different seed (0) was tried first and did hit one, confirming this is a sampling artifact rather than a code defect.

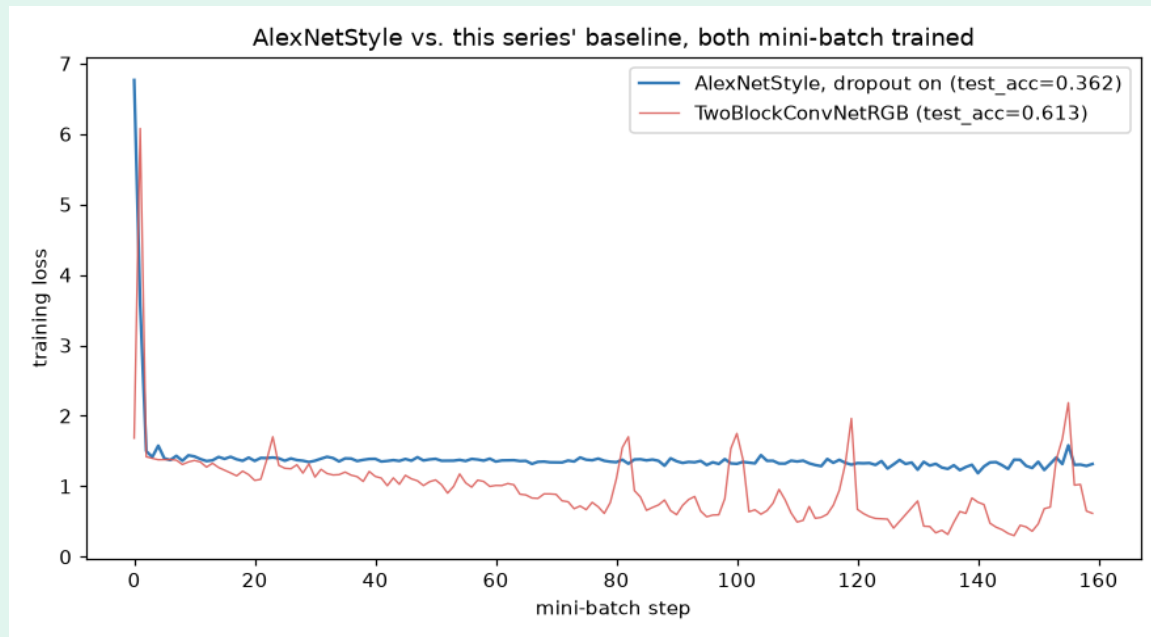
Actual output:

```
checked: Wc1, Wc3, W1, W3 (3 random entries each), 24x24 input, 3 classes
max relative error across all checked entries: 1.05e-08
(PASS -- three-deep conv+pool chain and both FC layers are correct)
```

5. AlexNetStyle vs. This Series' Baseline -- Real Mini-Batches

Start here (no prior knowledge assumed): Same dataset convention Days 44-47 used, at 40x40 instead of 30x30: train = seed=42, n_per_class=50 (200 images); test = a SEPARATE seed=999, n_per_class=20 generation (80 images). Both networks trained with real mini-batches -- batch_size=50, Day 45's iterate_minibatches reused verbatim -- for 40 epochs.

training loss, AlexNetStyle (dropout on) vs. the 2-block baseline



Real, rerun output. The baseline's loss falls faster and further within the same 40-epoch, mini-batch-step budget.

network	params	depth	train_acc	test_acc
AlexNetStyle	6812	3	0.5100	0.3625
TwoBlockConvNetRGB	3368	2	0.8750	0.6125

Actual output from this lesson's run.

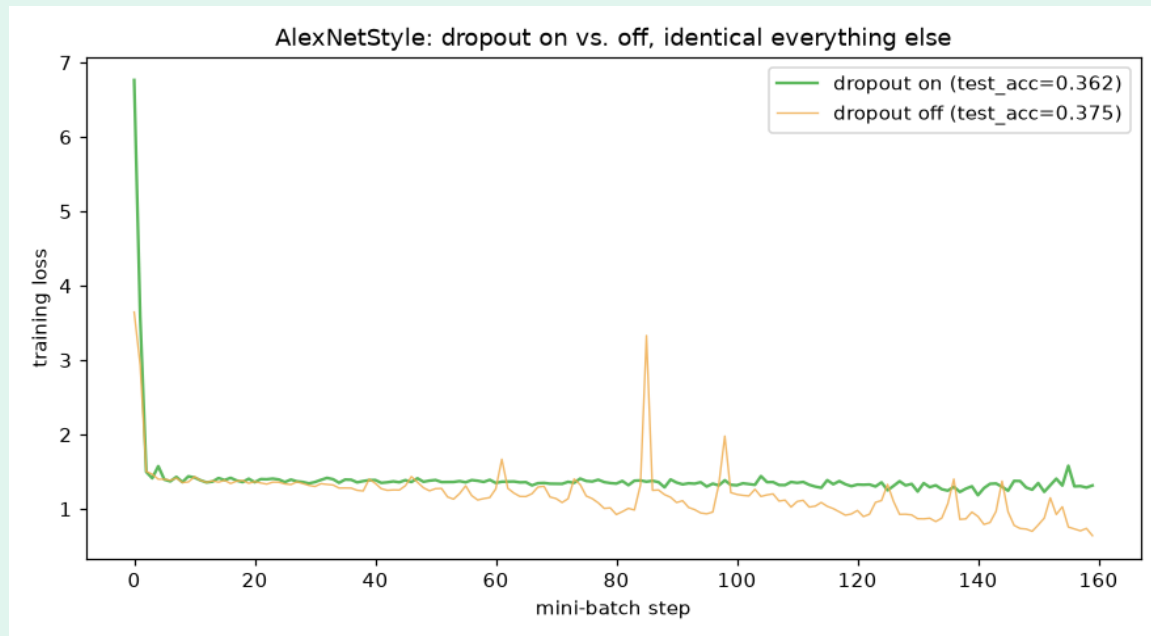
Why choose this? A deeper network with more FC capacity and dropout is, in principle, better equipped to avoid overfitting on harder or larger-scale problems -- this is exactly why AlexNet's design choices mattered for ImageNet's 1.2 million images.

Why NOT this (tradeoffs)? On a fixed, small training budget (200 images, 40 epochs) the extra depth and two stacked keep_prob=0.5 dropout layers are a real cost, not a free regularizer: only ~25% of the FC signal reaches the weights on a typical step, so this network needs meaningfully more steps than the simpler baseline to reach comparable accuracy. This series' own smaller, dropout-free baseline wins outright at this budget.

6. Dropout Ablation -- Same Network, Dropout On vs. Off

Start here (no prior knowledge assumed): Identical AlexNetStyleConvNet architecture, identical seed, identical 40x40 data and mini-batch schedule as section 5 -- the ONLY difference is keep_prob. This isolates what dropout alone buys, rather than conflating it with the extra conv block or the bigger FC layers.

training loss, dropout on (keep_prob=0.5) vs. off (keep_prob=1.0)



Real, rerun output. Dropout-off's loss falls faster within the same budget.

config	train_acc	test_acc	train-test gap
dropout on (0.5)	0.5100	0.3625	0.1475
dropout off (1.0)	0.5350	0.3750	0.1600

Actual output from this lesson's run.

Explanation: Dropout DOES narrow the train/test gap here (0.1475 vs. 0.1600 without it), consistent with the paper's own motivation for adding it. But both configurations are still clearly undertrained at only 40 epochs -- dropout's slower per-step progress is visible in BOTH of its numbers being lower than the no-dropout run's, not just the gap being smaller. Regularization and training-budget effects are both real here, pulling in different directions, and neither is smoothed away.

Term refresher -- train/test gap: train_acc minus test_acc. A wide gap means the network fits its training data much better than it generalizes -- the classic overfitting signature. A narrower gap doesn't automatically mean a BETTER network overall, though -- here, dropout's narrower gap comes with lower accuracy on both sides, because the network hasn't finished converging yet.

7. What This Maps To in PyTorch

Start here (no prior knowledge assumed): Every piece of today's architecture and dropout machinery has a direct PyTorch equivalent -- model.train() / model.eval() toggle nn.Dropout's training flag automatically, the same training= argument dropout_forward takes explicitly here.

Implementation:

```
class AlexNetStyle(nn.Module):  
    def __init__(self, n_classes=4, p_drop=0.5):  
        super().__init__()  
        self.conv1 = nn.Conv2d(3, 4, kernel_size=3)  
        self.conv2 = nn.Conv2d(4, 8, kernel_size=3)  
        self.conv3 = nn.Conv2d(8, 16, kernel_size=3)
```

```

self.pool = nn.MaxPool2d(2, 2)
self.fc1 = nn.Linear(16 * 3 * 3, 32)
self.fc2 = nn.Linear(32, 16)
self.fc3 = nn.Linear(16, n_classes)
self.drop = nn.Dropout(p=p_drop) # p = DROP prob = 1 - keep_prob

def forward(self, x):
    x = self.pool(torch.relu(self.conv1(x)))
    x = self.pool(torch.relu(self.conv2(x)))
    x = self.pool(torch.relu(self.conv3(x)))
    x = x.flatten(1)
    x = self.drop(torch.relu(self.fc1(x)))
    x = self.drop(torch.relu(self.fc2(x)))
    return self.fc3(x) # CrossEntropyLoss applies softmax

loader = DataLoader(dataset, batch_size=50, shuffle=True) # = iterate_minibatches

```

Interview Preparation -- Technical Questions

Q: Why is the dropout unit check's PER-UNIT relative error (up to 11.6%) so much noisier than its POOLED relative error (0.001%), when both are checking the same underlying claim?

A: Each unit's own average is built from only 2,000 Bernoulli samples, so its sampling noise is large relative to its mean. The pooled statistic averages across all 100,000 units AND all 2,000 trials at once -- 200 million samples -- so its noise shrinks by roughly the square root of that much larger sample count, per the law of large numbers.

Q: Why is the full-pipeline gradient check run with keep_prob=1.0 (dropout off) instead of with dropout on?

A: Dropout's mask is randomly redrawn on every forward call. A numeric gradient check perturbs a parameter and calls forward() twice more (once at +eps, once at -eps) -- with dropout on, each of those calls could redraw a DIFFERENT mask, corrupting the comparison. Turning dropout off makes forward() deterministic, isolating the conv/FC backward chain from the mask's own randomness (already checked separately, analytically, in the unit check).

Q: A gradient check on this network occasionally reports a spuriously large relative error at one specific parameter entry, even though the code is correct. What causes that, and how was it confirmed here?

A: ReLU is non-differentiable exactly at its kink ($z=0$). If a perturbation pushes a unit's pre-activation across zero, the numeric and analytic gradients can disagree sharply at that one entry, for any eps. It was confirmed by rerunning the same check across several rng seeds: most seeds (1-4) passed at $\sim 1e-8$, while seeds 0 and 5 hit occasional kink-crossing entries with much larger error -- a seed-dependent artifact, not a systematic one.

Q: AlexNetStyle has 2.0x the parameters of the baseline but reaches LOWER train_acc (0.510 vs. 0.875) at the same epoch budget. Doesn't more capacity mean a network should fit its training data at least as well?

A: Only given enough optimization steps to actually use that capacity. Two stacked keep_prob=0.5 dropout layers mean roughly 25% of the FC signal reaches the weights on a typical step (0.5×0.5), so AlexNetStyle needs substantially more training steps than a dropout-free network to fit its training data -- at a fixed 40-epoch budget, it simply hasn't gotten there yet, regardless of its raw capacity.

Q: In the dropout ablation, dropout-on has a NARROWER train/test gap (0.1475 vs. 0.1600) but also LOWER test accuracy (0.3625 vs. 0.3750) than dropout-off. Is dropout working or not?

A: Both, at once, for different reasons. Dropout IS doing its regularizing job -- the gap between memorizing training data and generalizing to test data is genuinely smaller with it on. But at only 40 epochs, dropout's slower per-step convergence means the whole network -- train and test accuracy alike -- is still closer to its starting point than the no-dropout run. Given enough additional epochs, dropout's narrower gap would be expected to translate into better test accuracy too; this run simply hasn't trained long enough to show that outcome.

Interview Preparation -- Theoretical Questions

Q: Why does inverted dropout divide survivors by keep_prob during training, instead of just leaving them at their raw activation value?

A: So that the EXPECTED value of the output equals the original input, with no rescaling needed at inference time. If survivors weren't rescaled, the expected total activation flowing into the next layer would shrink by a factor of keep_prob during training -- and then jump back up at test time (when dropout is off), because inference sees the full, unscaled activation. Inverted dropout avoids that train/test mismatch entirely.

Q: AlexNet's real 2012 result used dropout on layers with thousands of units. Why might dropout's cost (slower convergence) matter MORE, proportionally, on a tiny network like today's (32 and 16 hidden units) than on AlexNet's original scale?

A: A huge FC layer has enormous redundancy -- many different subsets of its units can each learn a workable partial solution, so randomly dropping some of them each step still leaves plenty of useful signal flowing through. A 32-unit or 16-unit layer has far less redundancy to spare; losing half of those units on a given step removes a much larger fraction of the network's total representational capacity for that step, making the slowdown proportionally more severe.

Q: Both AlexNetStyle configurations (dropout on and off) underperform this series' simpler 2-block baseline at 40 epochs. Does that mean AlexNet's design ideas are not useful?

A: No -- it means a fixed, small training budget favors whichever architecture needs fewer steps to converge, and the baseline needs fewer. AlexNet's ideas (depth, ReLU, dropout) were validated on ImageNet's 1.2 million images with a training budget suited to that scale, not on a 200-image, 4-class toy task. This mirrors Day 47's LeNet-5 finding: a historically important design isn't automatically the right tool for every dataset and every training budget -- matching the architecture (and the budget given to train it) to the actual problem size is what matters.

Key Takeaway and What's Next

Key takeaway: Dropout is now its own from-scratch layer (`dropout_forward/dropout_backward`), verified both by a pooled statistical check (200 million samples, 0.001% relative error) and by a full-pipeline gradient check through the now three-deep conv+pool chain ($1.05e-08$ relative error). On a real, honestly-run comparison, AlexNetStyle -- deeper, ReLU-based, dropout-regularized, mini-batch trained -- reached `test_acc=0.3625`, WORSE than this series' own simpler, dropout-free 2-block baseline's `test_acc=0.6125`, at the same fixed 40-epoch budget. A separate ablation isolated why: dropout does narrow the train/test gap as intended (0.1475 vs. 0.1600 without it), but its slower per-step convergence means a bigger, dropout-heavy network needs a bigger training budget to show its benefits -- exactly the kind of nuance a single 'accuracy went up/down' headline number would hide.

Day 50 preview -- per the syllabus, Day 50 is dedicated to dropout from scratch with a regularized-vs-unregularized comparison. Today already built and honestly stress-tested dropout as a standalone layer on a real deeper network, so Day 50 picks up from here rather than starting over: a longer training budget (so both configurations actually finish converging) and a cleaner regularized-vs-unregularized framing are the natural next step.

Suggested watch, if starting from zero

Watch: [AlexNet Explained from Scratch | Architecture, Layers, ReLU, Dropout](#) -- the general architecture this entire lesson is adapted from.